

Course Outline: Training a Regression Model

Title: Training a Regression Model **Skill:** __ — Skill x: Model selection and training **Learnpack:** + Cómo entrenar un modelo de Regresión (predice un número) **File:** s__-w17d50-como-entrenar-un-modelo-de-regresion-pre **Prerequisite:** Training a Classification Model (train/test split, scikit-learn fit/predict, evaluation concepts) **Target Audience:** Beginner developers who just trained a classifier and now need to predict continuous values **Total Lessons:** 8 **Format:** Mixed (12.5% hands-on)

Course Description

A regression model predicts a number — price, temperature, score, duration — instead of a category. This course builds directly on the classification training learnpack: the train/test split pattern, the scikit-learn API, and the concept of evaluation metrics are already known. What changes is the type of output (continuous instead of categorical), the algorithms available, and the metrics used to measure success. Students train their first regressor, then learn to interpret regression-specific metrics to decide whether the model is good enough.

Course Philosophy

This course emphasizes:

- Building on what's already known — no re-introduction of concepts covered in the classification learnpack
 - Understanding why regression metrics differ from classification metrics
 - Developing intuition for what a "good" regression score actually means in practice
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Regression Algorithms	3
02 - Regression Metrics	2
03 - Assessment	1
04 - Outro	1
Total	8

00.0 From Categories to Numbers 

Learning Objectives:

- Explain what makes regression different from classification
- Connect the classification training workflow to regression — identifying what stays the same and what changes

Content Outline:

- Classification recap: predicting which category something belongs to
- Regression: predicting a numeric value on a continuous scale
- What stays the same: train/test split, scikit-learn API, the concept of evaluation
- What changes: output type, algorithm choices, evaluation metrics
- Real-world regression problems: house prices, temperature forecasts, delivery times, sales revenue

Transitions: This lesson sets the frame. Section 01 covers the algorithms available for regression and how to train them.

Key Principles:

- Regression and classification are both supervised learning — the pipeline is identical, only the output type changes
- A regression model's output has infinite possible values; a classifier's output is one of a fixed set of labels

Examples:

- Predicting a house price given its features (area, location, bedrooms) — the output is €185,000, not "expensive"
- Predicting how many minutes a pizza delivery will take — the output is 23, not "fast" or "slow"

01.0 Regression Algorithms

Learning Objectives:

- Name the main families of regression algorithms available in scikit-learn
- Describe how each algorithm generates a numeric prediction
- Preview what lessons 01.1 and 01.2 cover (choosing and training)

Content Outline:

- Linear Regression: fits a straight line (or hyperplane) to the data — the simplest and most interpretable
- Decision Tree Regressor: same tree-splitting logic as the classifier, but the leaf outputs an average value instead of a class label
- Random Forest Regressor: ensemble of decision tree regressors — same ensemble logic, different output type
- How regression algorithms generate numbers: instead of voting on a class, they average or interpolate to produce a value
- Overview: 01.1 covers how to choose between these algorithms; 01.2 puts theory into practice

Transitions: This overview introduces the algorithm landscape. Lesson 01.1 goes deeper into which algorithm to use when.

Key Principles:

- Linear regression is the baseline — always start there; if it underperforms, try tree-based models
- The scikit-learn API is identical to classifiers: `.fit()`, `.predict()`, `.score()` — only the class name changes

Examples:

- Linear Regression on house prices: learns that each additional square meter adds €1,200 to the predicted price
 - Decision Tree Regressor: splits data into regions, then predicts the average price of all houses in that region
 - Random Forest Regressor: averages the predictions from 100 different trees
-

01.1 Choosing a Regressor

Learning Objectives:

- Choose between linear regression, decision tree regressor, and random forest regressor for a given scenario
- Explain the interpretability-accuracy tradeoff in regression
- Identify when linear regression will fail (non-linear relationships)

Content Outline:

- **Linear Regression:** best when relationship between features and target is roughly linear; interpretable coefficients tell you exactly how each feature affects the prediction; fast to train
 - When it fails: the relationship is non-linear (e.g., price grows exponentially with location quality)
- **Decision Tree Regressor:** handles non-linear relationships naturally; interpretable rules; overfits easily without depth limits
 - When to use: quick baseline for non-linear problems; when you need explainable rules
- **Random Forest Regressor:** robust, generally accurate for complex datasets; slower, less interpretable
 - When to use: production-grade predictions when accuracy matters more than interpretability
- Practical heuristic: start with Linear Regression → if R^2 is low, try Decision Tree → then try Random Forest

Transitions: Now that you know which regressor to reach for, lesson 01.2 trains one end-to-end and produces observable predictions.

Key Principles:

- The simplest model that achieves acceptable accuracy is always preferred

- Always train a baseline linear model first — it tells you the floor from which more complex models must improve

Anti-patterns:

- **Jumping to Random Forest without trying Linear Regression first:** complex models are harder to debug and may not outperform a simple baseline
- **Using regression for categorical outputs:** predicting "0 or 1" with a regressor instead of a classifier leads to nonsensical outputs like 0.73

Examples:

Scenario	Recommended Regressor	Why
Predicting fuel efficiency from car specs	Linear Regression	Physics-based linear relationships
Predicting house prices in a complex market	Random Forest	Many non-linear interactions
Predicting delivery time for a small dataset	Decision Tree	Interpretable, handles non-linearity

01.2 Training a Regressor

Challenge Prompt: Load the prepared dataset from `clean_data.csv`, split it into train and test sets (80/20, `random_state=42`), train a `LinearRegression` model, and print the R^2 score on the test set.

Solution Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv("clean_data.csv")
X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

r2 = model.score(X_test, y_test)
print(f"R² score: {r2:.4f}")
```

Challenge Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv("clean_data.csv")
X = df.drop("target", axis=1)
y = df["target"]

# Step 1: split the data (80% train, 20% test, random_state=42)
X_train, X_test, y_train, y_test = # your code here

# Step 2: create and train a LinearRegression model
model = # your code here
# your code here

# Step 3: print the R2 score on the test set
r2 = # your code here
print(f"R2 score: {r2:.4f}")
```

02.0 Regression Metrics

Learning Objectives:

- Explain why accuracy is meaningless for regression
- Define MAE, MSE, RMSE, and R² and describe what each measures
- Preview the interpretation guidance in lesson 02.1

Content Outline:

- Why accuracy doesn't work for regression: a prediction of 185,000 for a house worth 186,000 is excellent — but "wrong" in classification terms
- **MAE (Mean Absolute Error):** average absolute difference between prediction and actual value; same unit as the target; intuitive
 - MAE = 5,000 means "on average, predictions are off by €5,000"
- **MSE (Mean Squared Error):** squares the errors before averaging; penalizes large errors more heavily; harder to interpret directly
- **RMSE (Root Mean Squared Error):** square root of MSE; same unit as target; penalizes outliers more than MAE
- **R² (Coefficient of Determination):** proportion of variance in the target explained by the model; 1.0 is perfect, 0.0 means the model does no better than predicting the mean, negative means worse than the mean
- Overview: 02.1 goes into interpreting these scores in practice and recognizing overfitting in regression

Transitions: This lesson defines the metrics. Lesson 02.1 teaches how to read them — what scores are good, what signals a problem.

Key Principles:

- No single metric tells the full story — report at least R^2 and MAE together
- R^2 is unitless and context-free; MAE/RMSE are in the target's unit and tell you the real-world error magnitude

Examples:

- MAE of 2,000 on a dataset of house prices ranging from 50,000 to 500,000 is excellent; MAE of 2,000 on a dataset where all prices are between 15,000 and 20,000 is terrible
 - R^2 of 0.85 means the model explains 85% of the price variance — 15% is unexplained
-

02.1 Interpreting Your Regressor's Results

Learning Objectives:

- Interpret R^2 , MAE, and RMSE values in the context of a specific problem
- Identify overfitting in a regression model by comparing train and test scores
- Decide whether a regression model is good enough or needs improvement

Content Outline:

- **What is a good R^2 ?**
 - Depends on the domain: $R^2 = 0.7$ is excellent for social science data; $R^2 = 0.7$ may be too low for financial forecasting
 - Rule of thumb: above 0.8 is generally strong; below 0.5 suggests the model is missing important features
- **Reading MAE and RMSE in context:**
 - Always compare the error to the range of your target variable
 - $RMSE > MAE$ means the model makes some large errors — investigate outliers
- **Overfitting in regression:**
 - Train $R^2 = 0.98$, Test $R^2 = 0.61$ → the model memorized training data
 - Fix: reduce tree depth, add regularization (Ridge/Lasso for linear models), get more data
- **When to improve vs when to ship:**
 - If the error is acceptable for the business use case, stop tuning
 - If $R^2 < 0.5$, revisit features and data quality before changing algorithms

Transitions: With metrics understood, the assessment tests command of the full regression training and evaluation pipeline.

Key Principles:

- Overfitting looks the same in regression as in classification: train score >> test score
- The right threshold for "good enough" comes from the business problem, not a universal standard

Anti-patterns:

- **Reporting only R^2 :** an R^2 of 0.9 with huge MAE is possible (if variance is huge); always report actual error magnitude
- **Accepting negative R^2 :** a negative R^2 means the model is worse than predicting the mean — something is fundamentally wrong (data leakage, wrong target column, wrong split)

Examples:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R2: {r2:.4f}")
```

03.0 Regression Training Assessment 🧠

Learning Objectives:

- Demonstrate understanding of regression vs classification distinctions
- Apply correct metric selection and interpretation for regression problems
- Identify overfitting and choose appropriate responses

Question Format: Multiple choice

Questions:

1. What is the key difference between a regression model and a classification model?
 - a) Regression models use more data
 - b) Regression predicts a continuous numeric value; classification predicts a category
 - c) Regression models require normalized data; classification does not
 - d) Classification is supervised learning; regression is not
2. You train a Linear Regression model on housing data. $R^2 = 0.32$ on the test set. What is the most likely cause?
 - a) The model was correctly fit on test data too — results are reliable
 - b) The relationship between features and price is non-linear; linear regression is too simple
 - c) The data was not normalized properly
 - d) R^2 of 0.32 is excellent for housing data

3. Your regression model produces $MAE = 500$ on a dataset where target values range from 400 to 600. What does this mean?
- a) Excellent — 500 is a small error relative to the target range
 - b) The model is essentially useless — average error almost equals the full target range
 - c) Acceptable — MAE below 1000 is always fine
 - d) The model has overfitted to the training set
4. Which scikit-learn class would you use to train a regression model on a dataset with many non-linear feature interactions?
- a) `LogisticRegression`
 - b) `LinearRegression`
 - c) `RandomForestRegressor`
 - d) `DecisionTreeClassifier`
5. Train $R^2 = 0.97$, Test $R^2 = 0.54$. What does this indicate?
- a) The model is generalizing well — train and test scores should differ
 - b) The model is underfitting — it needs more complexity
 - c) The model is overfitting — it memorized training data and fails on new data
 - d) The test set is too small — results are not reliable
6. What does an R^2 of 0.0 mean?
- a) The model predicts perfectly
 - b) The model explains none of the variance — it performs no better than predicting the mean
 - c) The model has not been trained yet
 - d) The data has too many outliers for R^2 to be meaningful
7. Why should you report both R^2 and MAE instead of just R^2 ?
- a) R^2 alone can hide large real-world errors if the target variance is high
 - b) MAE is required by scikit-learn before scoring
 - c) R^2 is only valid for linear models; MAE works for all regressors
 - d) They always give the same information — reporting both is redundant

Evaluation Criteria:

- 7/7: Excellent — complete mastery of regression pipeline
- 5–6/7: Good — minor gaps in metric interpretation
- 3–4/7: Needs review — revisit sections 01 and 02
- Below 3/7: Restart recommended

Score Interpretation: Questions map to: regression vs classification (Q1), low R^2 root cause (Q2), MAE in context (Q3), algorithm selection (Q4), overfitting (Q5), R^2 of zero (Q6), metric completeness (Q7).

Content Outline:

- Course recap: what students accomplished
 - Understood how regression differs from classification (same pipeline, different output type)
 - Learned three regression algorithms and when to use each
 - Trained a Linear Regression model end-to-end
 - Interpreted MAE, RMSE, and R^2 in context
 - Identified overfitting in a regression model
- Connection to bigger picture: regression is one of the two fundamental supervised learning tasks — classification and regression together cover the majority of real-world ML use cases
- Next steps: the next modules cover model debugging (what to do when performance is poor) and data orchestration (keeping the pipeline running)
- Resources for further exploration:
 - scikit-learn documentation: [LinearRegression](#), [RandomForestRegressor](#), metrics
 - "Hands-On Machine Learning with Scikit-Learn" (Aurélien Géron) — regression chapters
 - Kaggle: housing price competitions are classic regression benchmarks
- Encouragement: predicting a number from raw data — with code you wrote from scratch — is the foundation of data science and ML engineering

Note: This is conclusion only — no theory, exercises, or assessment.
